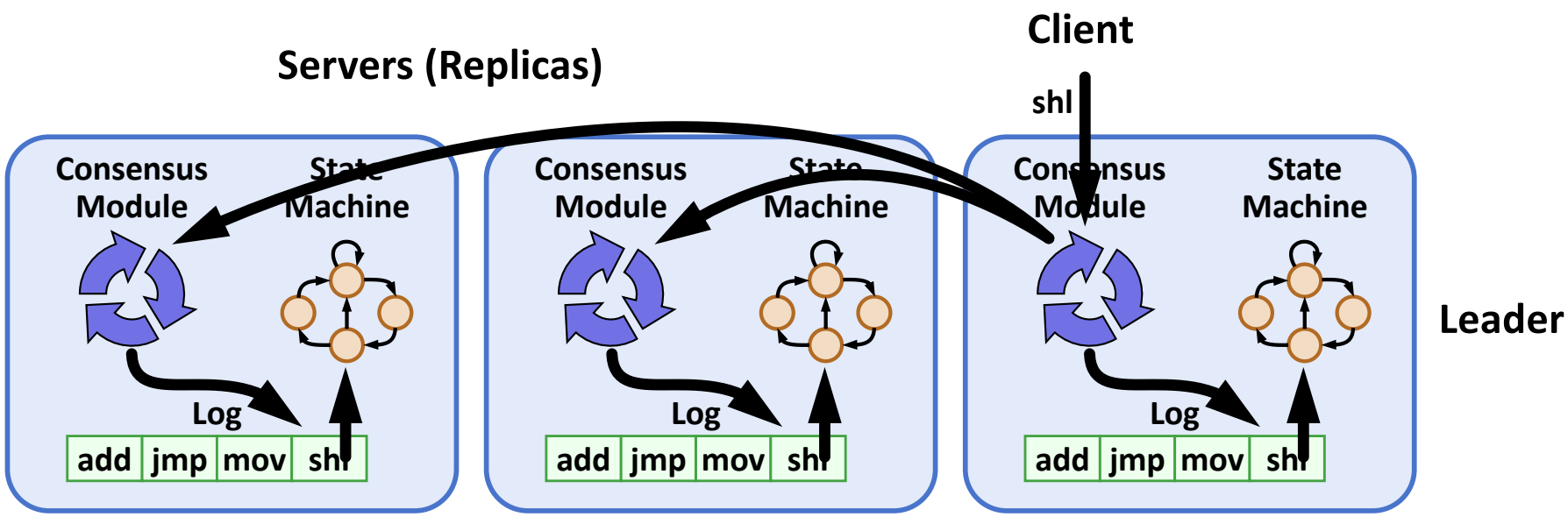
RSM Consensus Revisited: Bitcoin

Jeff Chase
Duke University

Replicated State Machine Consensus with a global log ("ledger")

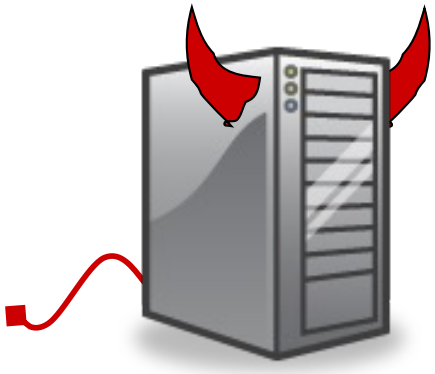


[Image from Raft presentation by Raft authors: Ongaro and Ousterhout]

Byzantine Fault Tolerance

- All systems studied we studied so far assume a **fail-stop** failure model.
- Byzantine fault tolerance (BFT): algorithms that persist in the face of evil.
  - Whom do you trust? Nobody. "**Trustless**"
  - Participants may lie, collude, or act unfaithfully (incorrectly).
  - These are **Byzantine** failures. (It's just a word.)
- Voting is suspect because attackers may create false identities ("sybils").

  → No leaders you can trust.

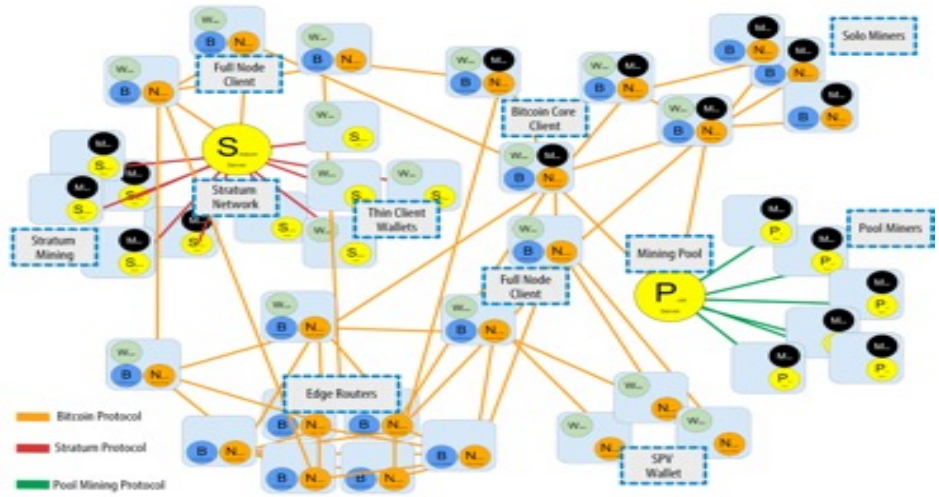


Principles of Bitcoin / Ethereum

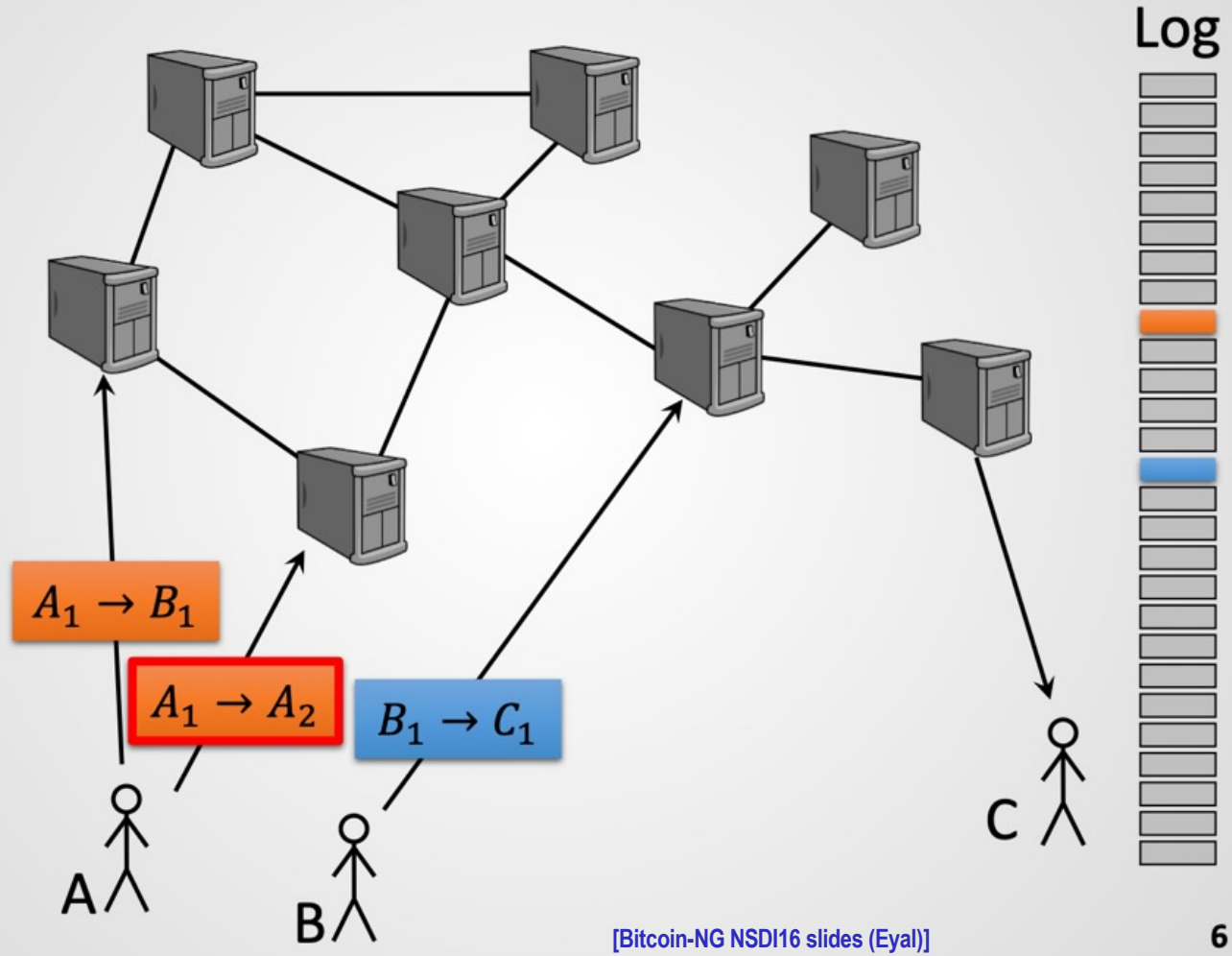
- **What is it?**
  - A digital currency. It is money: you can buy it for USD$, spend it for goods and services, or cash it in for USD$.
  - A family of programs and protocols, running on IP-connected computers that form a P2P network to manage the currency.
- **Who can join?**
  - Anyone with an IP-connected computer: just run a program.
  - You can remain anonymous (except for your IP address).
- **Who controls it?**
  - Nobody and everybody. It is a public and trustless system with clever built-in incentives for faithful behavior.
  - Even when some participants (e.g., “miners”) are large and powerful.

Broadcast overlay

- Broadcast ops and blocks
- Propagation delay?
- Metric: 90/90 time t
- After t seconds, 90% probability that 90% got it.



Blockchain: A Replicated State Machine



[Bitcoin-NG NSDI16 slides (Eyal)]

"Trustless" blockchains

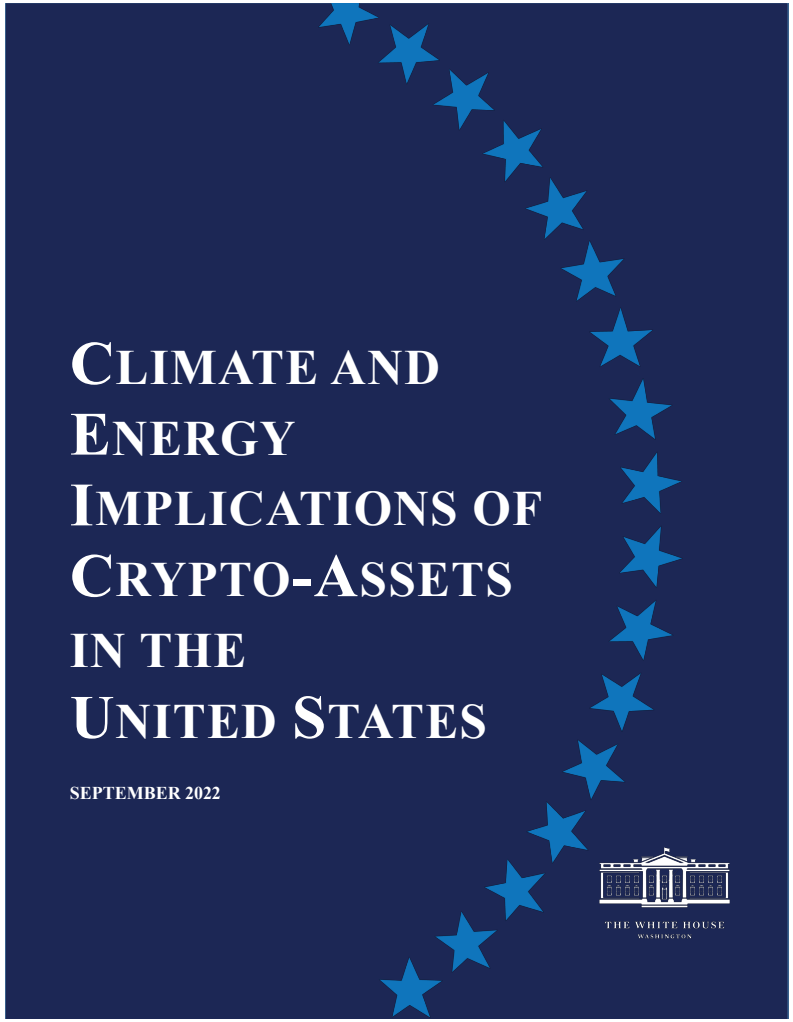
Bitcoin or "classic" blockchain:

- Public and open to anyone.
- Anyone can be a leader—but it takes work.
- Anonymous → an entity may control many sybil candidates for leader.
- Demands we select leaders "randomly" → PoW ("Nakamoto consensus")
  - (Weighted by computational power.)
- Everyone checks up on the leader(s) and votes.
- Apply incentives ($$$) for correct leader behavior.

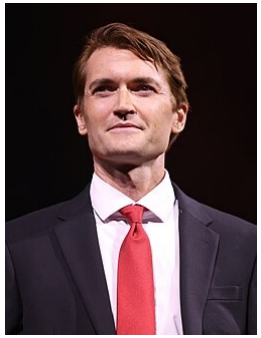
Alternative. Today many blockchain proposals use **permissioned** consensus in which a circle of leaders govern together with partial trust, accountability, and incentives for good behavior (compare: Ethereum **Proof-of-Stake**).

frankenstein.cs.wtf

PoW→Proof of Waste
est. ~200 TWh
globally in 2022



Crypto: why?



Dread Pirate Roberts
(Ross Ulbricht)
Silk Road Inc.

Dark web druglords charged with laundering $2.3M in Bitcoin

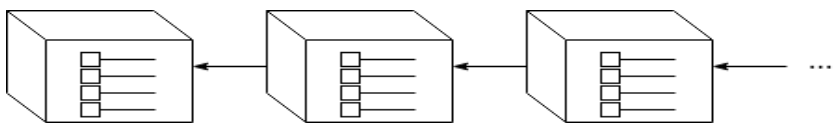
The Next Web • 2 hours ago

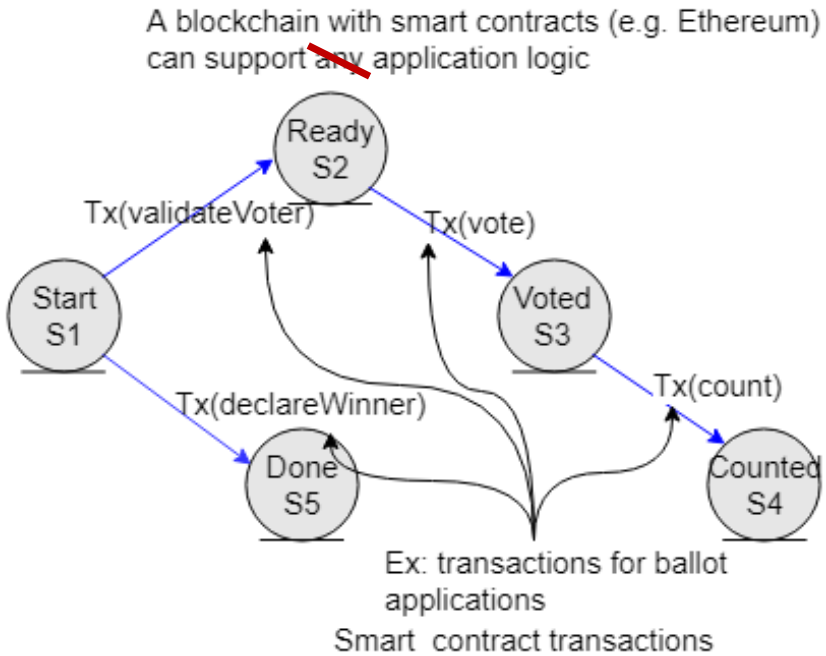
How a Dark Web Drug Ring Was Uncovered After Suspicious A.T.M. Withdrawals

Three New Jersey men were charged in Manhattan with setting up a virtual storefront in a hidden corner of the internet that sold illicit drugs.

Blockchain apps run as RSMs

- Bitcoin runs **one** service with a simple API: **transfer** funds.
- Ethereum: many user-supplied programs called **smart contracts** or **chaincodes** or **dapps**.
- Single timeline (ledger, chain)
- → Replicas converge





[stolen: original source unknown]

Principals of Bitcoin (or Ethereum)

- Q: How to name/identify "accounts" belonging to principal identities like Alice, Bob, etc.?
- We need an account address that enables anonymous proof of ownership.
  - Anyone can make up an account/address.
  - Nobody knows who owns the account.
  - The owner (or the owner's software) can prove on demand that it owns the account and the $$$.
  - If the owner transfers $$$ to another account, others in the network can verify the owner's intent.

Bitcoin: Accounts and their addresses

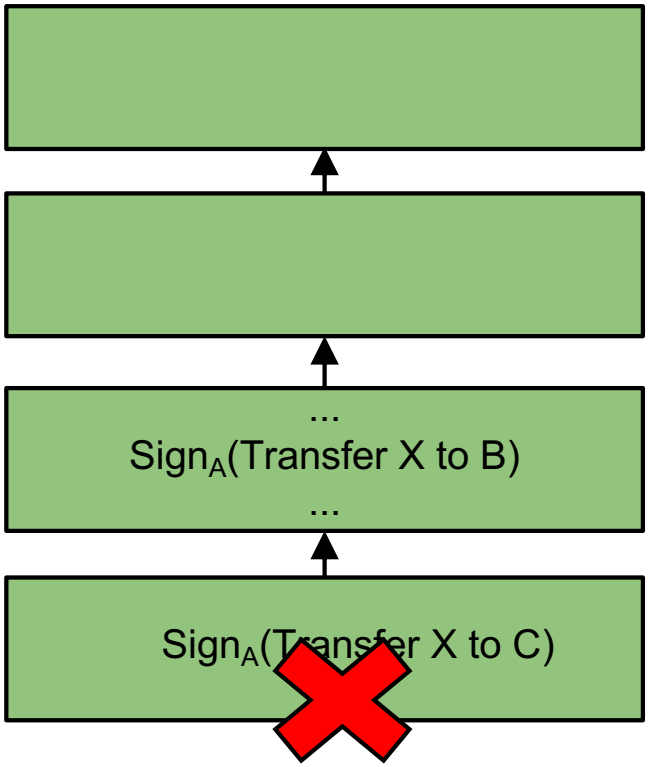
- An address names a principal: an entity that can own bitcoin.
- It is the hash of a public key.
- The account owner is anyone with the matching private key.
- The owner may spend bitcoin by publishing a **signed** statement (a "transaction") to **transfer** it to another address (account).
- **Who knows the private key?**

Crypto-coin: questions

How can we keep money safe in a network of anonymous participants?

- How to reach consensus about who owns the money?
  - Ownership is mutually exclusive (exactly one owner per unit).
- How to validate transactions to be sure that the spenders really own the money they are spending?
  - **Double spend**: attempt to spend the same money more than once.
- Where does money come from? "Fiat" currency is minted by a central authority, but none exists here.
- How to protect against attempts to counterfeit or steal money, tamper/rewrite history, or disrupt the network?

Solution: Maintain a global public append-only log

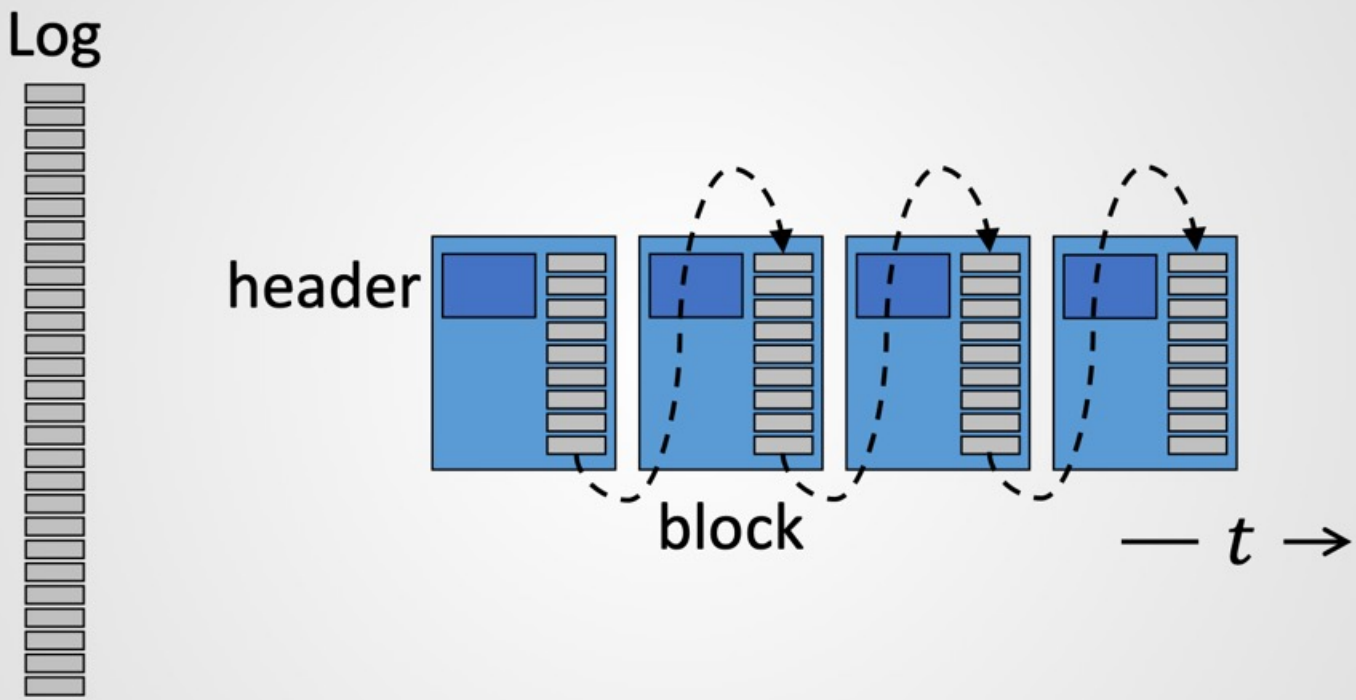


The block chain – a public ledger of all transactions.

(In Bitcoin, the log is extended in increments of blocks, each of which may contain thousands of transactions.*)*

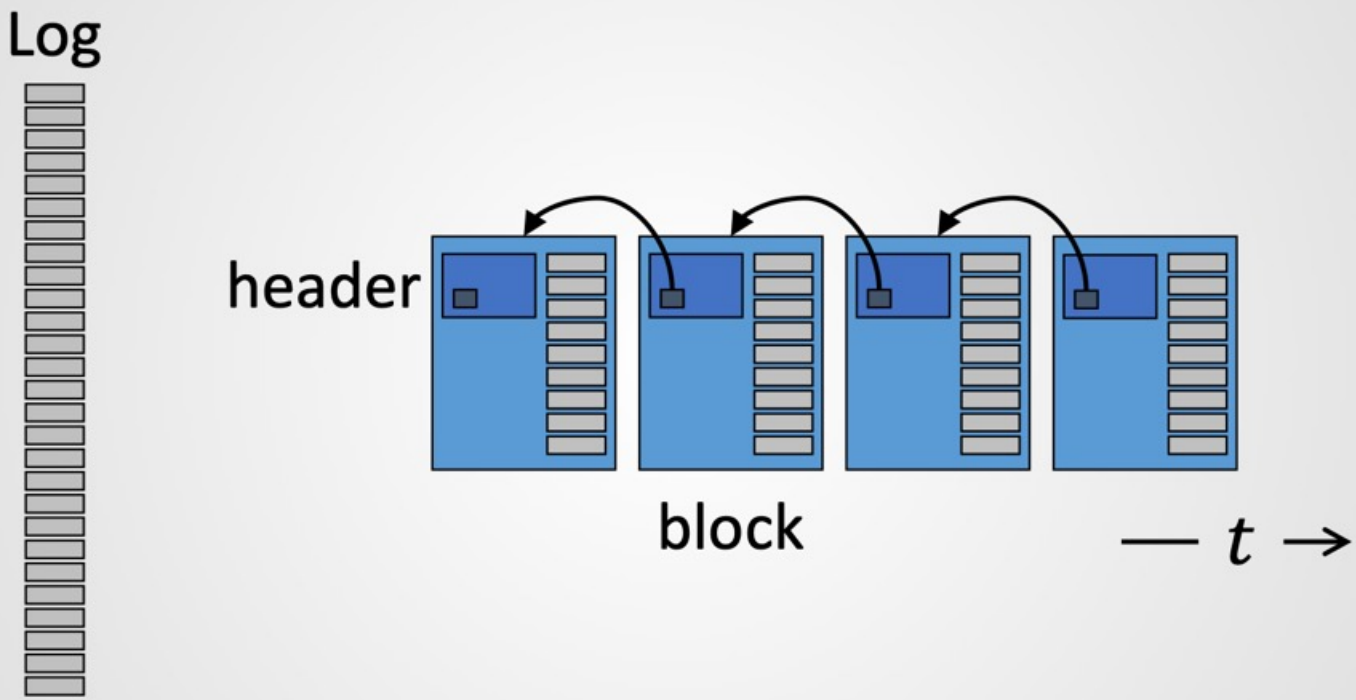
bmm

The Blockchain



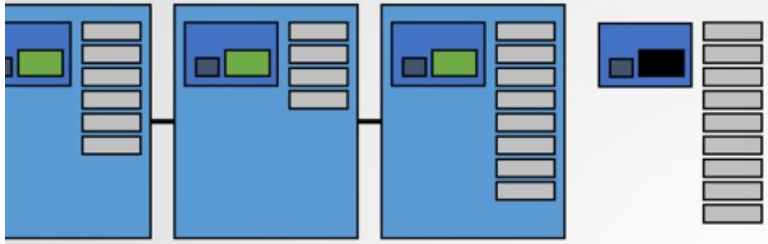
[Bitcoin-NG NSDI16 slides (Eyal)]

The Blockchain



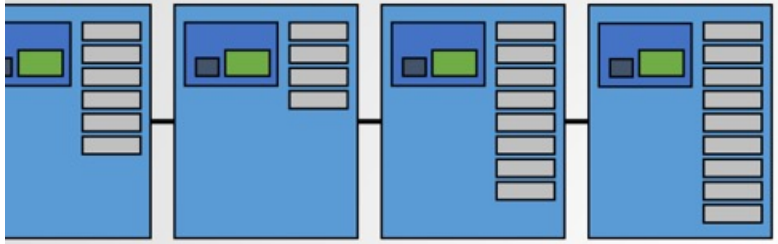
[Bitcoin-NG NSDI16 slides (Eyal)]

The Blockchain



[Bitcoin-NG NSDI16 slides (Eyal)]

The Blockchain



[Bitcoin-NG NSDI16 slides (Eyal)]

The Blockchain

Exponential, with
constant mean interval

[Bitcoin-NG NSDI16 slides (Eyal)]

Block chain: tamper-evident log

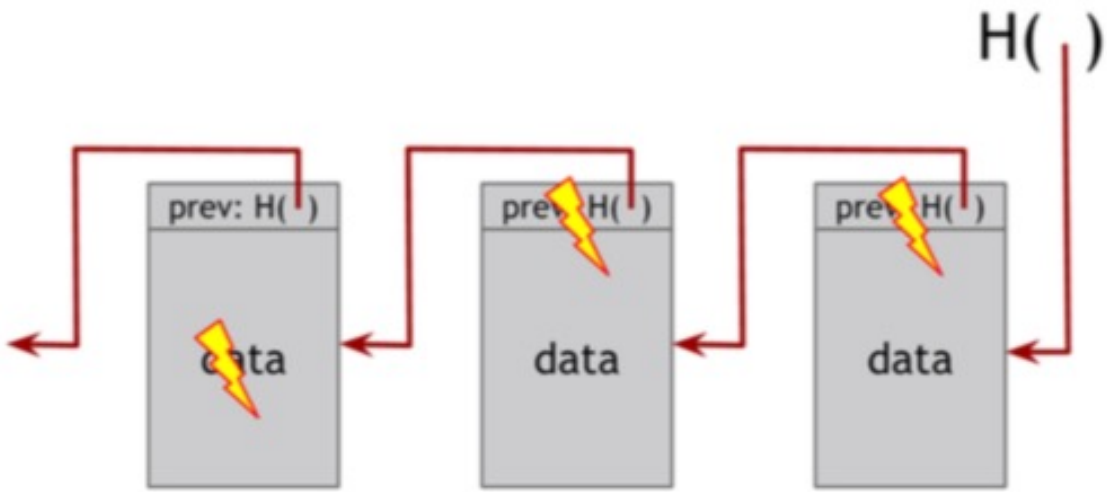


Figure 1.6 Tamper-evident log. If an adversary modifies data anywhere in the block chain, it will result in the next hash pointer being incorrect. If we store the head of the list, then even if the adversary modifies all of the pointers to be consistent with the modified data, the head pointer will be incorrect, and we will detect the tampering.

From Narayanan et. al.: Bitcoin and Cryptocurrency Technologies

Transfer

13.2 Transaction types and fields

Transactions are grouped into blocks (as detailed in Section 13.4), which are then linked to form the blockchain of Fig. 13.1. Here we consider the structure of individual Bitcoin transactions. For concreteness, we show representative data structures, reflecting relatively early implementations. While details will change over time (and already have),[2] this does not impact our main goal: introducing principal ideas and underlying concepts.

RELATING INPUTS TO OUTPUTS. A transaction transfers bitcoin from a sender (who owns the input, prior to the transaction) to a receiver (who will receive the input, as it turns into output). An input is itself the output of a prior transaction (Fig.13.4).

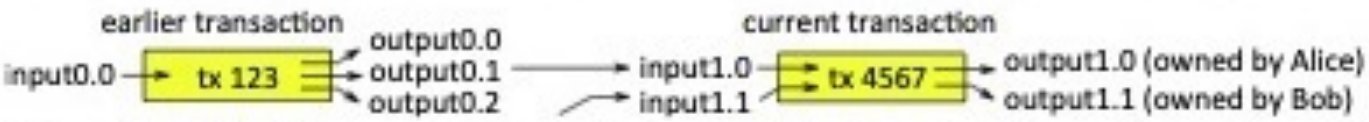


Figure 13.4: Model of how transaction inputs transition to outputs (becoming new inputs).

REGULAR TRANSACTION (DATA STRUCTURE). Table 13.1 shows the fields in a transaction with one input and one output. In general, there may be multiple inputs and outputs. A transaction's intended output value is explicitly stated (`btc-value`). The input value is implied, by reference to an earlier transaction (`prev-out-txID`). The total output

[2] For example, *SegWit* (BIP141, Section 13.9) moves unlocking scripts to the end of a transaction.

reference only

Section	Field Name	Description
top	txID†	(implicit) hash of rest of transaction; effectively, a serial no.
	version	protocol version (dictates block format and fields)
	n-in	number of inputs in this transaction
	n-out	number of outputs (indexed as: 0, 1, ...)
	lock-time	if 0, allows transaction to be included without delay
input		for each input, include:
	prev-out-txID	txID of earlier transaction whose output is to be used
	prev-out-index	which output of that earlier transaction (0 means first)
	scriptSig	an *unlocking script* (e.g., with signature to claim ownership)
output		for each output, include:
	out-index	(implicit) used as reference when this output is later spent
	btc-value	bitcoin value intended for receiver
	scriptPubKey	a *locking script* (e.g., with receiver's hashed public key)

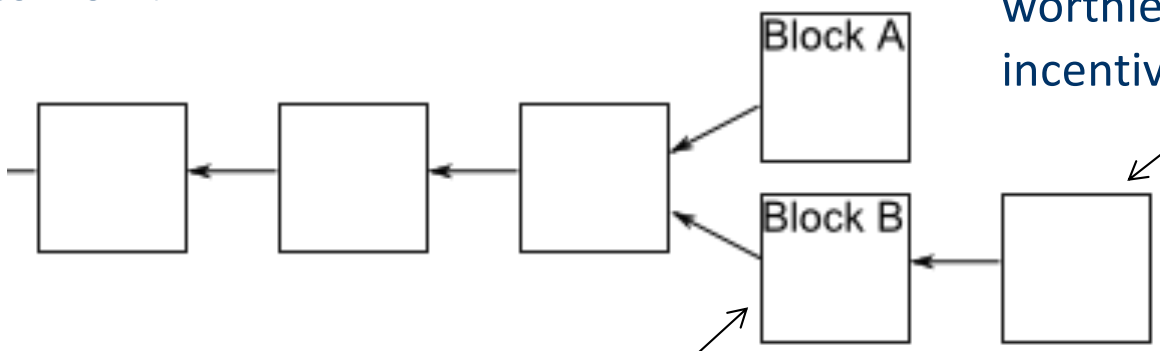
Table 13.1: Regular transaction (representative Bitcoin data structure). †txID is not an explicit field, but is computed by software on retrieving a transaction (and also used as an index to retrieve it). The other fields in the top section can be skipped over for now.

value must not exceed the total input (lest money be arbitrarily created). The hash of a transaction serves as its *transaction ID* (txID), and is how transactions are referenced. The input section's field prev-out-txID is the hash of a previous transaction.

Growing the log: adding a block

Anybody can be a leader (miner).
Anybody can produce a block, add it to the chain, and broadcast it.
But it takes work.

When a miner links to a block B, it accepts B as the head of the valid chain. If others disagree, then the miner's block is worthless ….so miners have an incentive to get it right!

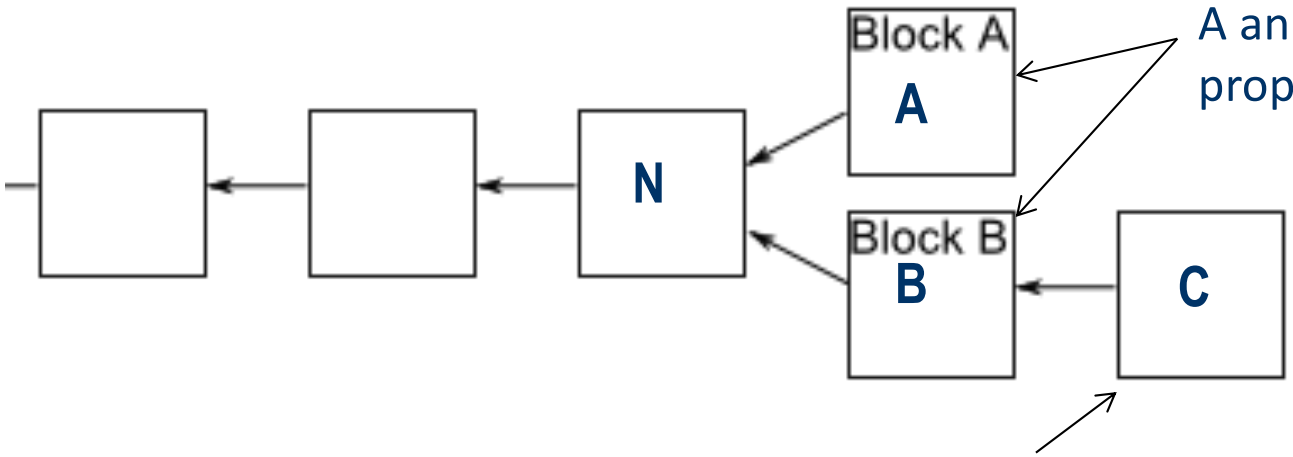


Other peers check each block and "vote" on it.
If block A is posted "too late" (e.g., block B is added first), or if block A is invalid, then other miners ignore it and build the chain in another direction.
The longest chain wins.

In this way Bitcoin achieves consensus on the ledger and the global history.

Forks and healing

- Forks result from faults, partitions, races, or attacks.
- Over time, forks should resolve: one branch grows longer (attracts more "votes") and so dominates.
- Consider successors to a proposed new block N:

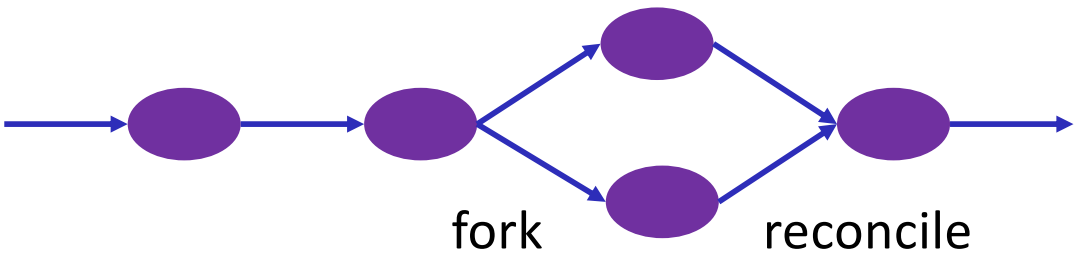


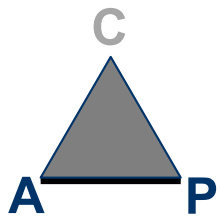
A and B accept N and each propose another new block.

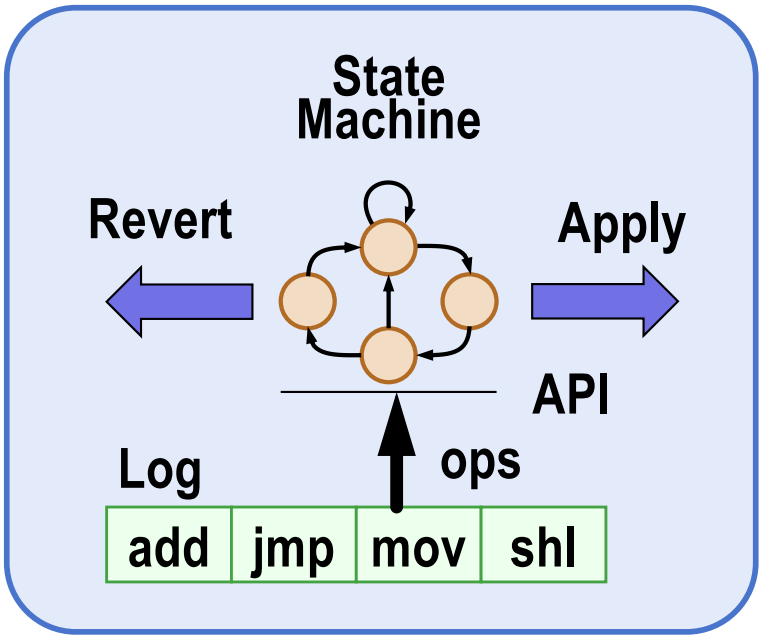
C ignores/rejects A and "votes" for the B chain with a new block C. All replicas that hear of block C (and approve it) adopt the B chain, reverting A.

AP consensus with a linear timeline

- Classic trustless blockchain systems are AP
- No concurrency control
- Transient forks may occur

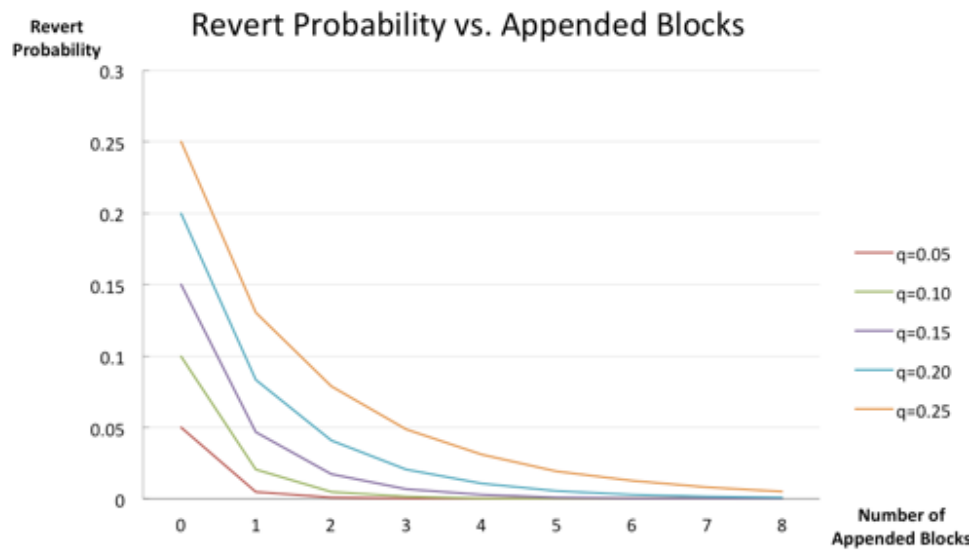






On a public blockchain, there is **no commit**: an op may revert due to a fork. Each replica rolls ops forward or back until the ledger stabilizes.

The case against blockchain



- Presumes Good > Evil.
- **Low throughput**. Current throughput for Bitcoin is 7 transactions/sec.
  Fix: *trust committee to pre-commit?*
- **Wastes energy.** Miners compute hashes all day in a race for money.
  Fix: *Proof-of-Stake*
- **Asynchronous probabilistic commit.** **Past** transactions/operations **could** revert.
- **Long confirmation latency**. Payee (or other observers) accept a transaction when the risk of it reverting falls below a threshold.

Can these challenges be fixed? Open research questions!

Coin miners

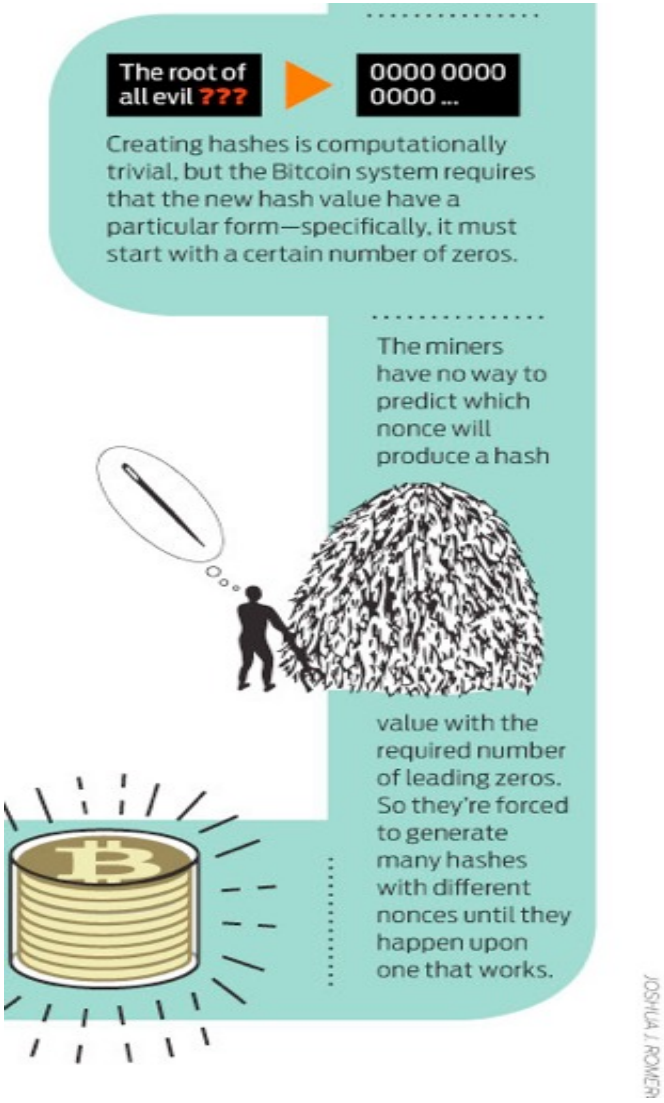
- Miners are replicas that hear transactions, validate them, and group them into blocks.
- They create/earn a bitcoin reward for each block they add to the chain ("mined").
- A miner must invest a large amount of compute power to solve a puzzle in order to find and publish a block. First miner wins!
  - Miners race to solve the puzzle and obtain the reward: probabilistic proof of work
- The reward is valid and spendable only if other nodes accept the published block.
  - Miners have an incentive to act faithfully and validate all transactions properly, so that other nodes "vote" to accept the block.

bitcoinminer.net

coincube.net

The miner's puzzle

- The miner computes a secure hash over the block (Merkle root), and includes the hash in the block.
- The puzzle: the miner must find a nonce for the block, such that the block's hash has a prefix of N zero bits.
  - N grows with time as computers get faster.
  - Probabilistic proof of work (PoW)
- Finding a nonce such that the first hash bit is zero is like flipping a coin: the odds are 50% for any random trial.
  - Each added zero bit reduces odds by a factor of 2.
- A miner's win probability is **proportional to its compute power**. In this way, the next miner to publish is selected "randomly" in a way that defends against sybils.



Proof-of-Work as a sybil defense

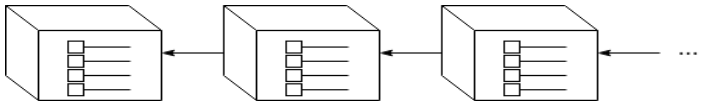
- A voting protocol is vulnerable to ballot stuffing. It is not sufficient to ignore "repeat votes" from a principal.
- Why? An attacker can appear to control many principals by minting new keypairs to vote with: a sybil attack.
- PoW addresses this problem by picking a miner for the next block randomly, with probability weighted by the miner's share of computational power in the replica pool.
- Problems: massively wasteful; presumes that Good controls more computational power than Evil.
  - Vulnerable to 51% attack in which an attacker controls the majority of participating computational power.

Global consensus in a blockchain

1. "Somebody" (miner) proposes a new block with a sequence of new transactions, appended to the ledger/log/chain as they view it.
   - Each block links at an index in the chain (block height), chosen by its creator and expressed as a signed hash link to the preceding block.
2. "Everybody" verifies for themselves that the new block is valid and is linked to the "end of the chain". Else reject block.
3. Each replica exposes its current view of the chain to clients, but its view may change if it hears of a different/longer log suffix.
4. If a replica accepts (votes for) a block, it also accepts the linked predecessor(s), overwriting/reverting its log suffix as necessary.

"Taller is better."

Longest valid chain wins.



Summary: how public blockchain differs from fail-stop consensus

1. No fixed configuration: replica group is open+large.
2. Nobody trusts anybody. No leader! 51% evil→failure.

   → No RPC: no trusted server to respond! Broadcast.
3. Log is public: public ledger. Anyone can read it.
   - Op confirmation appears in the log, so client and others see it.
4. *Replicas "take turns" as leader and check each other.
   - Leaders generate log entries in batches called blocks.
   - Leaders sign their blocks and chain hashes: no tampering.
5. Clients use keypairs to sign ops (entries, transactions).
6. No commit! Allows log forks! Repair→rollback.
   - Not immutable! Risk of reverting "confirmed" op declines with time.

Clearing and confirming ops

- A client requests an op (transaction) by broadcasting it to random other nodes, who pass it along by gossip.
- Eventually the miners hear the request and include it in their blocks (if the request is valid).
- Broadcast the new blocks in the same way.
- Transmission is similar to “anti-entropy” or “gossip”.
  - Nodes exchange hashes for objects they have seen: the recipient requests/forwards the data if it has not seen it before.
- A client considers its transaction confirmed when it appears in the blockchain with “enough” successor blocks.
  - Successor blocks act as confirming votes. But it could still revert! How many votes are enough?

Merkle tree [R. Merkle 1979]

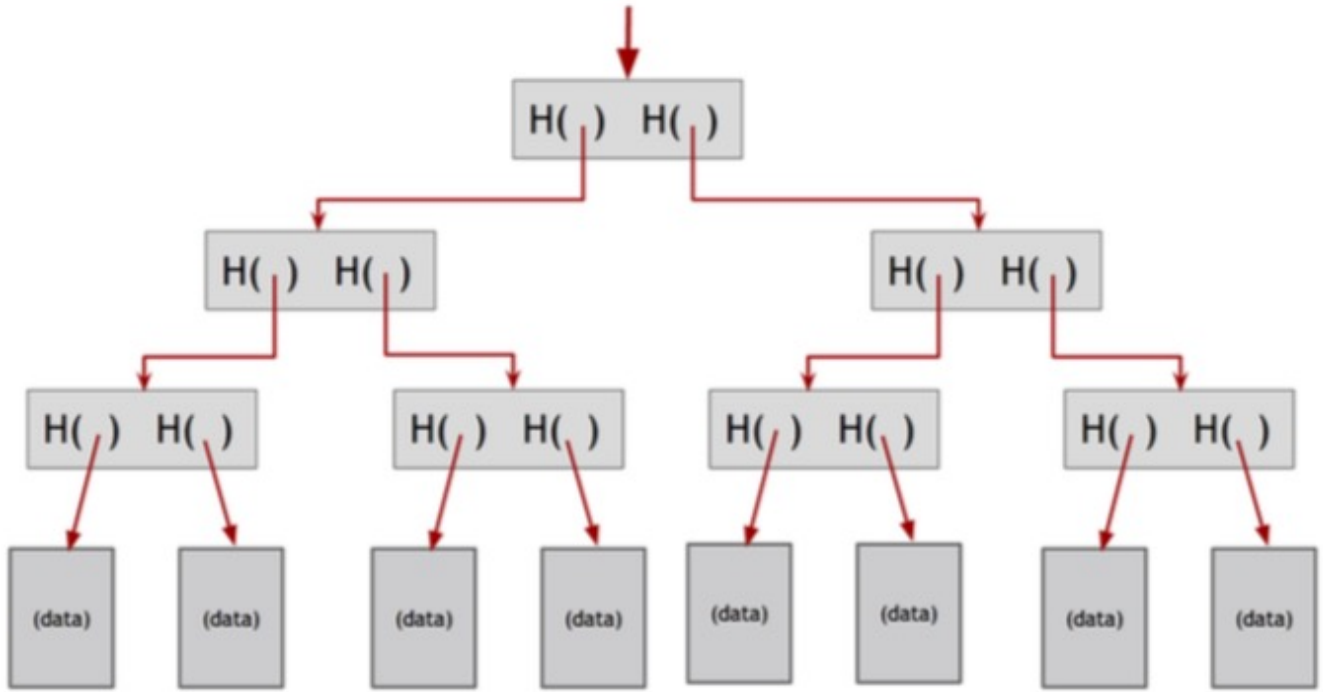


Figure 1.7 Merkle tree. In a Merkle tree, data blocks are grouped in pairs and the hash of each of these blocks is stored in a parent node. The parent nodes are in turn grouped in pairs and their hashes stored one level up the tree. This continues all the way up the tree until we reach the root node.

From Narayanan et. al.: Bitcoin and Cryptocurrency Technologies